

Generics in Go

Contents

1. [Why they exist](#)
2. [Basic syntax](#)
3. [Type parameters and constraints](#)
4. [The any and comparable predeclared constraints](#)
5. [The cmp.Ordered constraint \(Go 1.21+\)](#)
6. [Type sets and underlying types](#)
7. [Constraints with methods](#)
8. [Type inference](#)
9. [Generic types](#)
10. [The slices and maps packages](#)
11. [Common patterns](#)
12. [Generics vs interfaces](#)
13. [Performance](#)
14. [Limitations and gotchas](#)
15. [Best practices](#)
16. [When to choose what](#)
17. [Common gotchas](#)
18. [Quick reference](#)

Type parameters landed in Go 1.18. The syntax is small, the semantics are constrained, and the tradeoffs against interfaces matter. Here's the practical picture.

1. Why they exist

Before 1.18, writing a function that worked on any type meant either:

- Copy-pasting one per type (`MaxInt` , `MaxFloat` , `MaxString`).
- Using `interface{}` and reflection (slow, ugly, no type safety).
- Code generation (`go generate` with a template).

Generics let one function or type work over a set of types, with compile-time type safety and no boxing.

```
// before
func MinInt(a, b int) int { if a < b { return a }; return b }
func MinFloat(a, b float64) float64 { if a < b { return a }; return b }
func MinString(a, b string) string { if a < b { return a }; return b }

// after
func Min[T cmp.Ordered](a, b T) T { if a < b { return a }; return b }
```

2. Basic syntax

A function declares type parameters in square brackets between the name and the regular parameter list.

```
func Map[T, U any](s []T, f func(T) U) []U {
    out := make([]U, len(s))
    for i, v := range s {
        out[i] = f(v)
    }
    return out
}
```

Reading it:

- `T` and `U` are type parameters. Both have the constraint `any`.
- `s []T` is a slice of `T`. `f func(T) U` is a function from `T` to `U`. Return type is `[]U`.

Call it:

```
doubled := Map([]int{1, 2, 3}, func(n int) int { return n * 2 })
names := Map([]User{u1, u2}, func(u User) string { return u.Name })
```

The compiler infers `T` and `U` from the arguments. You can also write them explicitly:

```
Map[int, int]([]int{1, 2, 3}, double)
```

3. Type parameters and constraints

A type parameter without a constraint isn't allowed. The minimum constraint is `any` (alias for `interface{}`).

```
func F[T]           // compile error
func F[T any]       // OK; T can be any type
```

A constraint is an interface that specifies what operations the type must support.

```

type Number interface {
    int | int8 | int16 | int32 | int64 |
    uint | uint8 | uint16 | uint32 | uint64 |
    float32 | float64
}

func Sum[T Number](s []T) T {
    var sum T
    for _, v := range s { sum += v }
    return sum
}

```

The `|` is union: “any of these types.” Inside the function, you can use any operation that’s valid for every type in the union. `+` is valid for all numbers, so `sum += v` compiles.

4. The any and comparable predeclared constraints

Constraint	Meaning
<code>any</code>	Any type (alias for <code>interface{}</code>)
<code>comparable</code>	Any type where <code>==</code> and <code>!=</code> are defined

`comparable` includes numbers, strings, booleans, pointers, channels, interfaces, and arrays/structs of comparable types. It excludes slices, maps, and functions.

```

func Contains[T comparable](s []T, target T) bool {
    for _, v := range s {
        if v == target { return true }
    }
    return false
}

Contains([]int{1, 2, 3}, 2)           // OK
Contains([]string{"a", "b"}, "b")    // OK
Contains([][]int{{1}}, []int{1})     // compile error: []int not comparable

```

5. The cmp.Ordered constraint (Go 1.21+)

For types where `<`, `<=`, `>`, `>=` are defined:

```

import "cmp"

func Max[T cmp.Ordered](a, b T) T {
    if a > b { return a }
    return b
}

```

`cmp.Ordered` is the union of all integer types, all float types, and `string`. Before Go 1.21, you wrote this yourself or used `golang.org/x/exp/constraints.Ordered`.

6. Type sets and underlying types

A constraint like `int | string` is satisfied by `int` and `string` exactly.

A constraint like `~int | ~string` is satisfied by `int`, `string`, and any named type whose underlying type is `int` or `string`.

```
type Celsius float64
type UserID int

func F[T ~int | ~float64](x T) { ... }

F(UserID(1))           // OK because UserID's underlying type is int
F(Celsius(98.6))       // OK because Celsius's underlying type is float64
```

Use `~` (called the “approximation token”) when you want to accept user-defined types built on basic types. Without it, `Celsius(1.0)` would not satisfy `T int | float64`.

7. Constraints with methods

A constraint can include method requirements, like any other interface:

```
type Stringer interface {
    String() string
}

func PrintAll[T Stringer](s []T) {
    for _, v := range s { fmt.Println(v.String()) }
}
```

You can mix type sets and methods, but it’s rare to need both.

```
type IntStringer interface {
    ~int
    String() string
}
```

This means: an `int`-like type that also has a `String()` method.

8. Type inference

The compiler tries to figure out type parameters from the arguments:

```
func Pair[T, U any](a T, b U) struct{ A T; B U } {
    return struct{ A T; B U }{a, b}
}

p := Pair(1, "two")           // T=int, U=string
p := Pair[int, string](1, "two") // explicit, same result
```

Inference works when:

- Type parameters appear in regular parameter types (most cases).
- The return type alone is not enough; the inputs must determine the types.

```
func Zero[T any]() T { var z T; return z }

z := Zero[int]() // must specify; no way to infer from no arguments
```

For chains of generic functions, sometimes you need to spell types explicitly to help inference. The error message usually tells you which one is ambiguous.

9. Generic types

A struct, slice alias, map alias, or channel alias can be generic:

```
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }

func (s *Stack[T]) Pop() (T, bool) {
    if len(s.items) == 0 {
        var zero T
        return zero, false
    }
    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
}

s := Stack[int]{}
s.Push(1); s.Push(2)
v, ok := s.Pop()
```

Notes

- The receiver type includes the parameters: `(s Stack[T])`, not `(s Stack)`.
- Methods can use the type parameter (`T` here), but methods cannot declare new type parameters of their own. This is a deliberate limitation.

Why methods can't have their own type parameters

```
type Container[T any] struct{}

func (c *Container[T]) Map[U any](f func(T) U) Container[U] { ... }           //
                                     compile error
```

The reason: this would require runtime method dispatch on type parameter, which Go's implementation doesn't support. Workaround: make `Map` a package-level function:

```
func MapContainer[T, U any](c Container[T], f func(T) U) Container[U] { ... }
```

10. The slices and maps packages

Go 1.21 added two stdlib packages built entirely with generics. Replaces a dozen hand-rolled utilities.

slices

```
import "slices"

slices.Sort(s)                                // in-place sort (ordered T)
slices.SortFunc(s, func(a, b T) int { return ... })
slices.Contains(s, x)                         // comparable T
slices.ContainsFunc(s, predicate)
slices.Index(s, x)
slices.IndexFunc(s, predicate)
slices.Min(s); slices.Max(s)                  // ordered T
slices.Reverse(s)
slices.Clone(s)
slices.Equal(a, b)
slices.Concat(a, b, c)
slices.Insert(s, i, x...)
slices.Delete(s, i, j)
```

maps

```
import "maps"

maps.Keys(m)           // returns iter.Seq; collect with slices.Collect
maps.Values(m)
maps.Clone(m)
maps.Copy(dst, src)
maps.Equal(a, b)
```

In Go 1.23+, `maps.Keys` and `maps.Values` return iterators (`iter.Seq`). Use `slices.Collect(maps.Keys(m))` to get a `[]K`.

11. Common patterns

Map / Filter / Reduce

```
func Map[T, U any](s []T, f func(T) U) []U {
    out := make([]U, len(s))
    for i, v := range s { out[i] = f(v) }
    return out
}

func Filter[T any](s []T, pred func(T) bool) []T {
    out := s[:0] // reuse if mutating, or make a fresh slice
    for _, v := range s {
        if pred(v) { out = append(out, v) }
    }
    return out
}

func Reduce[T, U any](s []T, initial U, f func(U, T) U) U {
    acc := initial
    for _, v := range s { acc = f(acc, v) }
    return acc
}
```

Note

Go style still leans toward explicit `for` loops for clarity. Use these when they genuinely help; don't replace every loop with a chained `Map().Filter().Reduce()`.

Set type

```
type Set[T comparable] map[T]struct{}

func NewSet[T comparable]() Set[T] {
    return Set[T]{}
}

func (s Set[T]) Add(v T) { s[v] = struct{}{} }
func (s Set[T]) Has(v T) bool { _, ok := s[v]; return ok }
func (s Set[T]) Delete(v T) { delete(s, v) }
func (s Set[T]) Len() int { return len(s) }
```

Generic result type

```
type Result[T any] struct {
    Value T
    Err error
}

func Try[T any](f func() (T, error)) Result[T] {
    v, err := f()
    return Result[T]{Value: v, Err: err}
}
```

Useful in some pipeline patterns, but most Go code prefers returning `(T, error)` directly.

Optional / Maybe

```
type Optional[T any] struct {
    value T
    set bool
}

func Some[T any](v T) Optional[T] { return Optional[T]{v, true} }
func None[T any]() Optional[T] { return Optional[T]{} }

func (o Optional[T]) Get() (T, bool) { return o.value, o.set }
```

Mostly used for API clarity over `(T, bool)` returns. Not idiomatic everywhere.

Generic LRU cache

```
type LRU[K comparable, V any] struct {  
    cap int  
    m    map[K]*list.Element  
    list *list.List  
}  
// methods omitted
```

Generic two-type-param caches and maps are where generics shine.

12. Generics vs interfaces

Both let you write code that works on multiple types. Use generics when:

1. **You want to avoid boxing.** Putting an `int` into `any` allocates a heap box. Generics keep the `int` as an `int`.
2. **You want compile-time type safety on relationships.** A `Stack[int]` can only push and pop `int`. An `interface{}` stack can hold anything.
3. **You operate on the value with type-specific operations.** `+`, `<`, etc. work in a generic function bounded by `Number`; they don't work on `any`.
4. **The same algorithm applies to many types.** Sort, max, contains, etc.

Use interfaces when:

1. **You need runtime polymorphism.** Different types behave differently and you don't know which at compile time (e.g., `io.Reader` from file, network, gzip).
2. **You hold a heterogeneous collection.** A `[]Drawable` of shapes where each draws itself.
3. **Behavior matters more than shape.** Anything implementing `Read([]byte) (int, error)` is a `Reader`, regardless of what it is.
4. **Plugin or open-ended extensibility.** Third parties add new implementations later.

13. Performance

Generics avoid `any` boxing. A generic `Map[int, int]` operates on raw ints. An `any`-based map allocates an int box per element.

The compiler uses a strategy called GCShape stenciling: it generates one implementation per “shape” (broadly, types of the same size and pointer-ness share code). So:

- `Map[int, int]` and `Map[int32, int32]` share one compiled body.
- `Map[*Foo, *Bar]` shares with `Map[*Baz, *Qux]`.
- Each shape adds a small amount of binary size (~hundreds of bytes for typical functions).

Method calls inside a generic function go through a small per-type dispatch table for method-based constraints. Direct ops (+, <) are inlined per shape.

Net result: generics are roughly as fast as hand-written specialized code, often faster than interface-based code, and only slightly larger in binary size.

14. Limitations and gotchas

No methods with their own type parameters

```
func (c Container[T]) Map[U any](f func(T) U)           // compile error
```

Workaround: top-level function.

No type switching on type parameters directly

```
func F[T any](v T) {  
    switch v.(type) {           // doesn't work directly on T  
    case int: ...  
    }  
}
```

Workaround: convert to `any` first.

```
switch any(v).(type) {  
case int: ...  
}
```

Even better: rethink whether you should be generic. Type switching usually means you wanted an interface.

Can't construct a typed value directly inside a constrained function

```
func F[T Number](xs []T) T {  
    return T(1)           // OK for numeric types  
    return T{ }           // compile error: T is not a struct  
}
```

You can convert literals across numeric types if the constraint allows it. You cannot use struct literals because the constraint doesn't promise struct-ness.

Common workaround: declare `var zero T` and use that.

Constraint composition

Constraints are interfaces, and they can embed:

```
type SignedInt interface {  
    int | int8 | int16 | int32 | int64  
}  
  
type Comparable interface {  
    SignedInt | ~string  
}
```

Embedded interfaces become part of the type set via intersection.

No higher-kinded types

You can't write:

```
func F[M Mappable[T], T any]() M[T] { ... }
```

There's no way to abstract over "containers like Map" or "any type with one type parameter." Some functional patterns from Haskell/Scala don't translate.

Constants in generic functions

You can't define typed constants that depend on a type parameter inline. Use `var zero T` or pass values in.

15. Best practices

1. **Start without generics.** Add them when a real duplication appears. Premature generics is harder to read than three copies of one function.
2. **Use the standard `slices` and `maps` packages** instead of rolling your own.
3. **Use `cmp.Ordered` and `comparable`** for the most common constraints.
4. **Use `~` in constraints** when accepting user-defined types built on basics.
5. **Keep type parameters short and meaningful.** `T`, `K`, `V`, `E` are conventional.
6. **Type-parameter constraints belong above the function, not at the call site.** Don't make callers pass type witnesses; let the compiler infer.
7. **Don't make a generic version "just because."** Concrete is fine when there's one type.
8. **Convert to `any` for type assertions and reflection.** Don't fight the type system.
9. **Document the constraint's intent.** "T must be comparable because the function checks for duplicates" beats "T is comparable."
10. **Benchmark before assuming generics are faster.** They usually are vs `any`, but vs concrete code the difference is small.

16. When to choose what

Need	Choose
Need different behaviors for unknown types at runtime	interface
Need same algorithm for many types, compile-time	generics
Need to put heterogeneous values in one slot	any
Need polymorphism + new types added later	interface
Need to avoid boxing in a hot loop	generics
Need a plugin point with open extension	interface
Need a typed container (Stack, Queue, Cache)	generics
Need to wrap an unknown error	interface (error)
Need to write Min/Max/Sort/Filter once	generics

17. Common gotchas

Gotcha	Symptom	Fix
<code>T</code> without constraint	Compile error	Use <code>T any</code> minimum
Used <code>interface{}</code> for “any type”	Boxing overhead	Switch to <code>T any</code>
Forgot <code>~</code> on constraint	Named types don’t fit	Add <code>~</code> token
<code>comparable</code> includes interfaces with non-comparable values	Runtime panic on <code>==</code>	Be aware; use only when keys are simple
Method with extra type parameter	Compile error	Top-level function instead
Type switch on <code>T</code>	Compile error	Switch on <code>any(v).(type)</code>
Generic function passed where concrete signature needed	Type mismatch	Wrap in non-generic shim
Inference fails on return-only types	“cannot infer T”	Specify explicitly
Constraint embeds slice/map	Can’t constrain non-comparables	Use type set carefully
Big binary from too many specializations	Modest binary growth	Profile; usually fine
nil for <code>T any</code>	<code>var x T = nil</code> doesn’t compile	<code>var x T</code> (zero value); test with reflection if needed
Mixing constraints package versions	Conflicts with <code>cmp.Ordered</code>	Use stdlib <code>cmp</code> in Go 1.21+

18. Quick reference

What	Syntax
Generic function	<code>func F[T any](x T) T { ... }</code>
Multiple type params	<code>func F[T, U any](x T, y U)</code>
Constraint	<code>func F[T comparable](x T)</code>
Union constraint	<code>type N interface { int float64 }</code>
Underlying-type constraint	<code>~int ~float64</code>
Named constraint	<code>type N interface { ... }</code>
Ordered (Go 1.21+)	<code>cmp.Ordered</code>
Comparable	<code>comparable</code> (built-in)
Any	<code>any</code> (alias for <code>interface{}</code>)
Generic struct	<code>type Stack[T any] struct { items []T }</code>
Method on generic type	<code>func (s *Stack[T]) Push(v T)</code>
Explicit instantiation	<code>Map[int, string](xs, f)</code>
Inferred call	<code>Map(xs, f)</code>
Zero value of T	<code>var zero T</code>
Type switch on T	<code>switch any(v).(type) { ... }</code>
Slices package	<code>slices.Sort, Contains, Map, etc.</code>
Maps package	<code>maps.Keys, Values, Clone, Copy</code>
Constraint with method	<code>interface { String() string }</code>
Constraint embedding	<code>interface { OtherConstraint }</code>
Generic vs interface decision	Compile-time same algo => generics; runtime poly => interface